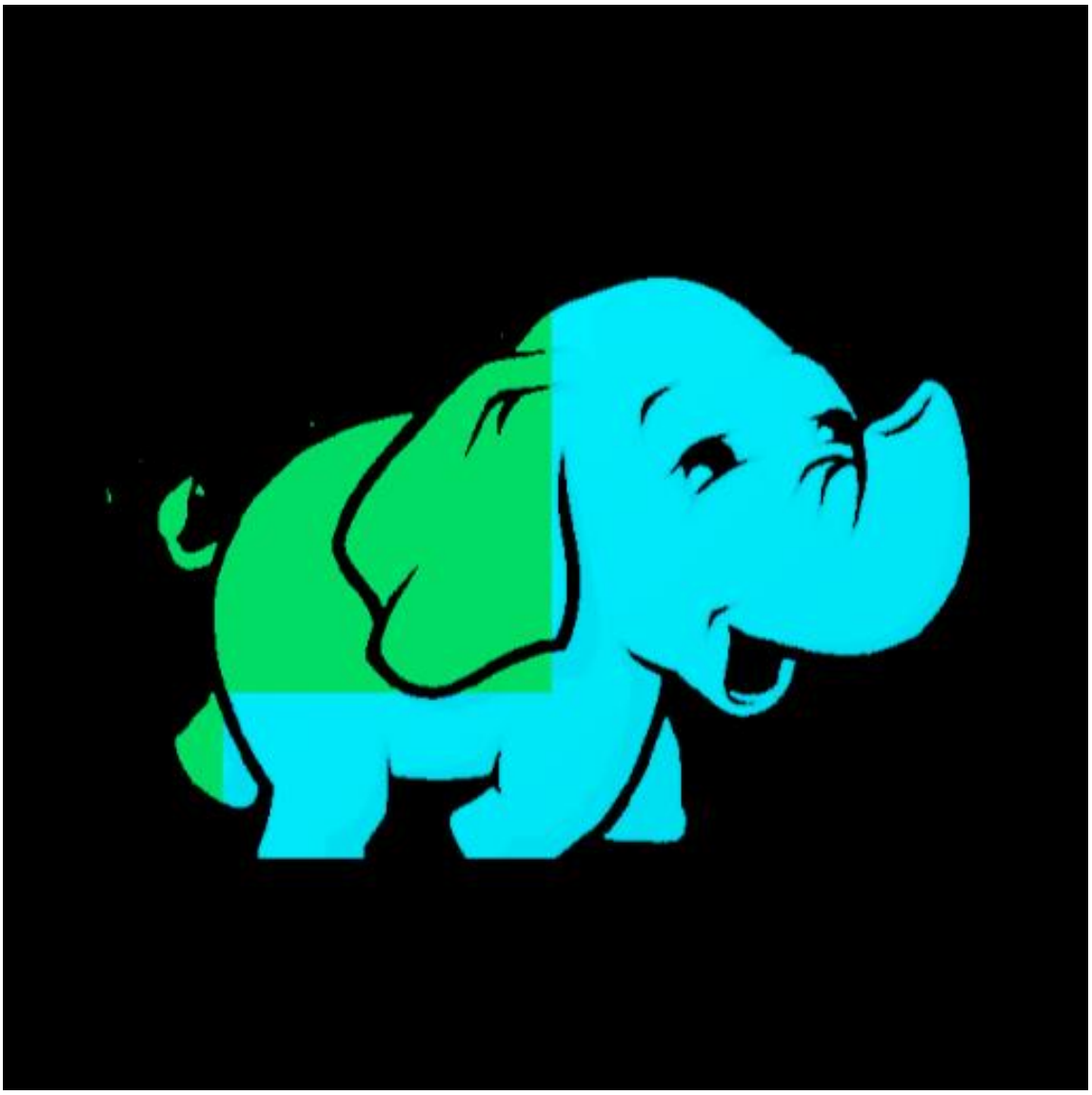
HADOOP:VERSION:1.0.1



DEVELOPER : Niranjan Kumar K

HADOOP : Simulator

PLATFORM : Browser Based

Start Now

HADOOP:VERSION:1.0.1

PreRequirements :

1. Browser (chrome/Firefox/Safari/..)
2. Internet (only once)

Environment Set Up :

Step-1 : Switch on internet and open browser.

Step-2 : Browse url https://kni1514.github.io/hadoop .

Step-3 : Install app !

Step-4 : Close browser and switch off internet !

Step-5 : Open app (Hadoop) !

Start Now

INDEX :

1.Installation and Configuration of Hadoop Cluster and Hadoop HDFS setup, NameNode & DataNode configuration.

2.Working with HDFS: File Operations Upload, read, delete, and replicate files in HDFS.

3.MapReduce Programming Basics Word count, sorting, and filtering examples in Java/Python.

4.Apache Hive & Pig for Querying Large Datasets Creation of tables, data loading, and running queries

5. Apache Spark Basics: RDDs and DataFrames Implement Spark transformations and actions

6. Data Preprocessing and Machine Learning using PySpark MLlib Classification or regression using MLlib pipelines

7 to 10 : cloud

Experiment 1: Installation and Configuration of Hadoop Cluster

Aim

Start Now

HADOOP:VERSION:1.0.1

To install and configure a Hadoop cluster in **Single-Node** and to configure **Hadoop HDFS** with the **NameNode and DataNode** for distributed storage and processing.

Description

Apache Hadoop is an open-source framework used for storing and processing large datasets in a distributed environment. **HDFS (Hadoop Distributed File System)** provides distributed storage by dividing files into blocks and storing them across DataNodes.

In a Hadoop cluster:

- **NameNode** manages the HDFS namespace and metadata.
- **DataNode** stores the actual data blocks.
- **Single-node mode** runs Hadoop services on one machine.

Procedure

Single-Node Hadoop Installation

1. Verify : python –version

```
[[ K : ~ ]] : $ python --version

Python 3.11.3 (Pyodide WebAssembly Engine)
```

2.Configure the Hadoop files:

   - core-site.xml
   - hdfs-site.xml
   - mapred-site.xml
   - yarn-site.xml

2. Configure core-site.xml to specify the default HDFS filesystem.

Start Now

3. Configure hdfs-site.xml to define the NameNode and DataNode storage directories.

```
[[ K : ~ ]] : $ cd hadoop/etc/hadoop/

[[ K : ~/hadoop/etc/hadoop ]] : $ ls

-rw-r--r--  core-site.xml
-rw-r--r--  hdfs-site.xml
-rw-r--r--  mapred-site.xml
-rw-r--r--  yarn-site.xml

[[ K : ~/hadoop/etc/hadoop ]] : $ #defaultly all configured !
```

4. Format the NameNode: hdfs namenode -format

```
[[ K : ~ ]] : $ hdfs namenode -format

26/09/02 15:00:00 INFO namenode.NameNode: STARTUP_MSG:
/************************************************************
STARTUP_MSG: Starting NameNode
STARTUP_MSG:   host = localhost/127.0.0.1
STARTUP_MSG:   version = 3.3.6
************************************************************/
26/09/02 15:00:01 INFO namenode.FSNamesystem: Formatting NameNode in
/home/Hacker/hadoop/data/dfs/name
26/09/02 15:00:01 INFO namenode.FSImage: Storage directory /home/Hacker/hadoop/data/dfs/name
has been successfully formatted.
26/09/02 15:00:01 INFO namenode.FSImage: Creating initial FSImage checkpoint with Cluster ID:
CID-8f92a10b-4e12-4c22-901a
26/09/02 15:00:01 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at localhost/127.0.0.1
************************************************************/
✓ NameNode formatted successfully. Ready for 'start-dfs.sh'.
```

5. Start the HDFS services: start-dfs.sh

```
[[ K : ~ ]] : $ start-dfs.sh

Starting namenodes on [localhost]
Starting datanodes on [datanode1.hadoop.local, datanode2.hadoop.local, datanode3.hadoop.local]
Starting secondary namenodes [localhost]
✓ HDFS daemons initialized successfully (NameNode: ACTIVE, DataNodes: 3/3 LIVE).
```

6. Verify the running Hadoop processes:

```
[[ K : ~ ]] : $ jps

18291 NameNode
18420 DataNode
18590 SecondaryNameNode
21045 Jps
```

7. Create a file :

```
[[ K : ~ ]] : $ vim k.txt
```

```
[Vim] "k.txt" saved and written.

[[ K : ~ ]] : $ cat k.txt

Hello Hacker !
```

8. Create a directory in HDFS:

```
[[ K : ~ ]] : $ hdfs dfs -mkdir Hacker

✓ Created HDFS directory: Hacker

[[ K : ~ ]] : $ hdfs dfs -ls

Found 1 items
drwxr-xr-x   - user supergroup          0 2026-09-02 09:45 Hacker
```

9. Upload a file into HDFS:

10. Verify the uploaded file:

```
[[ K : ~ ]] : $ cat k.txt

Hello Hacker !

[[ K : ~ ]] : $ hdfs dfs -put k.txt /Hacker/k.txt

✓ Copied from Local [k.txt] -> HDFS [/Hacker/k.txt] (3 replicas allocated across DataNodes)

[[ K : ~ ]] : $ hdfs dfs -ls /Hacker

Found 1 items
-rw-r--r--   3 user supergroup         14 2026-09-02 09:46 k.txt
```

11. Stop HDFS :

```
[[ K : ~ ]] : $ stop-dfs.sh

Stopping datanodes on [datanode1.hadoop.local, datanode2.hadoop.local, datanode3.hadoop.local]
Stopping namenodes on [localhost]
Stopping secondary namenodes [localhost]
✓ HDFS daemons stopped cleanly.
```

Output

The Hadoop cluster was successfully installed and configured. The **NameNode** was started as the master service, while the **DataNode(s)** were configured to store HDFS data blocks.

Example verification:

$ jps

NameNode

DataNode

SecondaryNameNode

The HDFS directory and uploaded file were also successfully created and verified.

Result

Thus, **Hadoop was successfully installed and configured in Single-Node cluster modes**. HDFS was configured with the **NameNode and DataNode**, and files were successfully stored and accessed using HDFS commands.

Experiment 2: Working with HDFS – File Operations

Aim

To perform basic file operations in **HDFS (Hadoop Distributed File System)** such as uploading, reading, deleting, and replicating files.

Description

HDFS file operations are performed in this experiment:

- Create directories
- Upload files
- View files
- Read files
- Delete files
- Change replication factor

Procedure

1. Start Hadoop Services

Start the HDFS services :

```
[[ K : ~ ]] : $ start-dfs.sh

Starting namenodes on [localhost]
Starting datanodes on [datanode1.hadoop.local, datanode2.hadoop.local, datanode3.hadoop.local]
Starting secondary namenodes [localhost]
✓ HDFS daemons initialized successfully (NameNode: ACTIVE, DataNodes: 3/3 LIVE).
```

Verify the running services:

HADOOP:VERSION:1.0.1

```
[[ K : ~ ]] : $ jps

18291 NameNode
18420 DataNode
18590 SecondaryNameNode
21045 Jps
```

---

2. Create a File

Create a sample file in the local system:

```
[[ K : ~ ]] : $ vim k.txt
```

```
[Vim] "k.txt" saved and written.

[[ K : ~ ]] : $ cat k.txt

Hello Hacker !
```

---

3. Create a Directory in HDFS

```
[[ K : ~ ]] : $ hdfs dfs -mkdir Hacker

✓ Created HDFS directory: Hacker

[[ K : ~ ]] : $ hdfs dfs -ls

Found 1 items
drwxr-xr-x   - user supergroup          0 2026-09-02 09:45 Hacker
```

---

4. Upload a File to HDFS

```
[[ K : ~ ]] : $ cat k.txt

Hello Hacker !

[[ K : ~ ]] : $ hdfs dfs -put k.txt /Hacker/k.txt

✓ Copied from Local [k.txt] -> HDFS [/Hacker/k.txt] (3 replicas allocated across DataNodes)

[[ K : ~ ]] : $ hdfs dfs -ls /Hacker

Found 1 items
-rw-r--r--   3 user supergroup         14 2026-09-02 09:46 k.txt
```

---

5. Read the File from HDFS

Start Now

```
[[ K : ~ ]] : $ hdfs dfs -cat /Hacker/k.txt

Hello Hacker !
```

---

6. Check File Information

```
 [[ K : ~ ]] : $ hdfs dfs -ls -h /Hacker/k.txt

 -rw-r--r--   3 user supergroup        14 2026-09-02 10:29 k.txt
```

---

7. Replicate the File

```
[[ K : ~ ]] : $ hdfs dfs -setrep -w 2 /Hacker/k.txt

Replication 2 set for /Hacker/k.txt

[[ K : ~ ]] : $ hdfs dfs -ls /Hacker

Found 1 items
-rw-r--r--   2 user supergroup        14 2026-09-02 10:31 k.txt
```

---

8. Delete the File from HDFS

```
[[ K : ~ ]] : $ hdfs dfs -rm /Hacker/k.txt

✓ Deleted /Hacker/k.txt

[[ K : ~ ]] : $ hdfs dfs -ls /Hacker

Found 0 items
```

---

9. Stop Hadoop Services

```
[[ K : ~ ]] : $ stop-dfs.sh

Stopping datanodes on [datanode1.hadoop.local, datanode2.hadoop.local, datanode3.hadoop.local]
Stopping namenodes on [localhost]
Stopping secondary namenodes [localhost]
✓ HDFS daemons stopped cleanly.
```

Output

The HDFS file operations were successfully performed.

The file was:

- Created in the local system.
- Uploaded to HDFS.
- Read from HDFS.
- Replicated using the replication factor.
- Deleted from HDFS.

Example:

$ hdfs dfs -ls /Hacker

Found 1 items

-rw-r--r-- 2 user supergroup 22 k.txt

Result

Thus, the basic **HDFS file operations such as upload, read, delete, and replication** were successfully performed using HDFS commands.

HADOOP:VERSION:1.0.1

Experiment 3: MapReduce Programming Basics

Aim

To implement basic **MapReduce programs** such as **Word Count, Sorting, and Filtering** using Python.

Description

MapReduce is a programming model used in Hadoop to process large amounts of data.

It consists mainly of two stages:

- **Mapper** – Processes the input data and generates key-value pairs.
- **Reducer** – Processes the key-value pairs and produces the final result.

In this experiment, Python is used to perform basic MapReduce operations.

Procedure

1. Create an Input File

```
[[ K : ~ ]] : $ vim k.txt
```

```
[Vim] "k.txt" saved and written.
[[ K : ~ ]] : $ cat k.txt
Hello Hacker !
```

---

2. Word Count

Start Now

HADOOP:VERSION:1.0.1

VIM - word_count.py — Press ESC to toggle INSERT / COMMAND modes (:wq to save & exit)

```
def wc(file_name):

    file_content = ""

    with open(file_name,"r") as f :
        file_content = f.read()

    return file_content.split(' ')
```

VIM - reducer.py — Press ESC to toggle INSERT / COMMAND modes (:wq to save & exit)

```
import word_count

def reduce(file_name):
    content = word_count.wc(file_name)

    words = list(set(content))

    dict = {}

    for w in words :
        dict[w] = content.count(w)

    return dict
```

VIM - mapper.py — Press ESC to toggle INSERT / COMMAND modes (:wq to save & exit)

```
import reducer

file_name = input()

words = reducer.reduce(file_name)

print(words)
```

```
[[ K : ~ ]] : $ python mapper.py k.txt

{'Hello': 1, 'Hacker': 1, '!': 1}
```

---

3. Sorting

Start Now

HADOOP:VERSION:1.0.1

```
VIM - sort.py                          Press ESC to toggle INSERT / COMMAND modes (:wq to save & exit)

import word_count

file_name = input()

words = sorted(word_count.wc(file_name))

print(words)
```

```
[[ K : ~ ]] : $ python sort.py k.txt

['!', 'Hacker', 'Hello']
```

---

4. Filtering

```
VIM - filter.py                        Press ESC to toggle INSERT / COMMAND modes (:wq to save & exit)

import word_count

file_name = input()

filter = input()

words = word_count.wc(file_name)

for w in words :
    if w.strip().startswith(filter) :
        print(w)
```

```
[[ K : ~ ]] : $ python filter.py k.txt H

Hello
Hacker

[[ K : ~ ]] : $ python filter.py k.txt He

Hello
```

Output

The MapReduce programs for **Word Count, Sorting, and Filtering** were successfully executed using Python.

Result

Start Now

Thus, basic **MapReduce operations such as Word Count, Sorting, and Filtering** were successfully implemented using Python.

Start Now

Experiment 4: Apache Hive & Pig for Querying Large Datasets

Aim

To create tables, load data, and perform basic queries on large datasets using **Apache Hive and Apache Pig**.

Description

Apache Hive is a data warehouse tool built on Hadoop. It allows users to store and analyze large datasets using **HiveQL**, which is similar to SQL.

Apache Pig is a data-flow platform used to analyze large datasets using a scripting language called **Pig Latin**.

In this experiment, Hive and Pig are used to:

- Create tables.
- Load datasets.
- Display data.
- Filter required records.
- Perform basic queries.

Procedure

A. Apache Hive

1. Start Hive

HADOOP:VERSION:1.0.1

```
[[ K : ~ ]] : $ hive

KSQL Database Engine Active (SQLite WebAssembly Local Engine)
type help, to see all commands
type exit or quit to return to Linux shell.

>>
```

2. Create a Table

```
>> create table hackers(id int,name text,rank int);

Query executed successfully.
```

3. Create an Input File

```
VIM - hackers.csv                    Press ESC to toggle INSERT / COMMAND modes (:wq to save & exit)

id,name,rank
101,Niranjan,1
102,Nikil,2
103,Nikilesh,3
104,Nihal,4
```

4. Load Data into the Table

```
>> load 'hackers.csv' into hackers;

Successfully loaded 4 records from "hackers.csv" into table "hackers".
```

5. Display the Data

```
>> select *from hackers;

id | name | rank
---------------
101 | Niranjan | 1
102 | Nikil | 2
103 | Nikilesh | 3
104 | Nihal | 4
```

6. Filter the Data

```
>> select name from hackers where rank = 1;

name
----
Niranjan
```

7. Find the Number of Hackers

Start Now

HADOOP:VERSION:1.0.1

```
>> select count(*) from hackers;

count(*)
--------
4
```

8.Store the Data

```
>> store hackers into hackers_back_up.csv" using pigstore(',')

Successfully stored 4 rows from table "hackers" into "hackers_back_up.csv".

>> exit

[KSQL] Exited KSQL shell mode. Returned to Linux shell.

[[ K : ~ ]] : $ cat hackers_back_up.csv

id,name,rank
101,Niranjan,1
102,Nikil,2
103,Nikilesh,3
104,Nihal,4
```

Output

The table was successfully created in Hive, data was loaded, and queries were executed. The dataset was also successfully loaded and processed using Pig.

Example output:

101,Niranjan,1

102,Nikil,2

103,Nikilesh,3

104,Nihal,4

Filtered output:

Niranjan

Result

Start Now

Thus, **Apache Hive and Apache Pig were successfully used to create tables, load datasets, and perform basic queries and data-processing operations on large datasets.**

Experiment 5: Apache Spark Basics – RDDs and DataFrames

Aim

To creating **RDDs and DataFrames** and implementing basic **transformations and actions** using **Apache Spark**.

Description

Apache Spark is an open-source distributed computing framework used to process large datasets quickly across multiple computers.

RDD (Resilient Distributed Dataset) is a fundamental data structure in Spark that stores distributed collections of data.

DataFrame is a distributed collection of data organized into named columns, similar to a database table.

In this experiment, Apache Spark is using for:

- Creating a Spark Session.
- Creating RDDs.
- Creating DataFrames.
- Performing transformations such as map(), filter(), and flatMap().
- Performing actions such as collect(), count(), and reduce().
- Displaying and processing datasets.

HADOOP:VERSION:1.0.1

Procedure

A. Apache Spark

1. Start PySpark

```
[[ K : ~ ]] : $ pyspark

Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   local PySpark-compatible shell
      /_/

Local Pyodide Python REPL with PySpark-compatible APIs.
SparkSession is available as 'spark'; SparkContext as 'sc'.
Type exit or quit to return to the terminal.

>>>
```

2. Import Required Libraries

```
>>> from pyspark.sql import SparkSession

>>> #creating spark session

>>> spark = SparkSession.builder.appName("MY_WORK_SPACE").master("local[*]").getOrCreate()
```

3. Create an RDD

```
>>> students = ["Niranjan","Nikil","Nikilesh","Nihal"]

>>> rdd = spark.sparkContext.parallelize(students)

>>> rdd.collect()

['Niranjan', 'Nikil', 'Nikilesh', 'Nihal']
```

4. Perform Map Transformation

```
>>> map_rdd = rdd.map(lambda x : x.lower())

>>> map_rdd.collect()

['niranjan', 'nikil', 'nikilesh', 'nihal']
```

5. Perform Filter Transformation

```
>>> df.filter(df.STUDENT_ID == "G7").show()

+------------+--------------+
| STUDENT_ID | STUDENT_NAME |
+------------+--------------+
| G7         | Nikilesh     |
+------------+--------------+
```

Start Now

6. Perform FlatMap Transformation

```
>>> lines = spark.sparkContext.parallelize(["hello python","hello spark"])
>>> lines.collect()
['hello python', 'hello spark']
>>> words = lines.flatMap(lambda x : x.split(' '))
>>> words.collect()
['hello', 'python', 'hello', 'spark']
```

7. Perform Actions

```
>>> rdd.collect()
['Niranjan', 'Nikil', 'Nikilesh', 'Nihal']
>>> rdd.first()
Niranjan
>>> rdd.count()
4
>>> rdd.take(2)
['Niranjan', 'Nikil']
```

B. Apache Spark DataFrames

8. Create a DataFrame

```
>>> data = [ ("G6","Nihal"),("G7","Nikilesh"),("G8","Nikil"),("G9","Niranjan")]
>>> columns = ["STUDENT_ID","STUDENT_NAME"]
>>> df = spark.createDataFrame(data,columns)
```

9. Display the Data

```
>>> df.show()
+------------+--------------+
| STUDENT_ID | STUDENT_NAME |
+------------+--------------+
| G6         | Nihal        |
| G7         | Nikilesh     |
| G8         | Nikil        |
| G9         | Niranjan     |
+------------+--------------+
```

10. Filter the Data

```
>>> df.filter(df.STUDENT_ID == "G7").show()

+------------+--------------+
| STUDENT_ID | STUDENT_NAME |
+------------+--------------+
| G7         | Nikilesh     |
+------------+--------------+
```

11. Select Required Columns

```
>>> df.select("STUDENT_NAME").show()

+--------------+
| STUDENT_NAME |
+--------------+
| Nihal        |
| Nikilesh     |
| Nikil        |
| Niranjan     |
+--------------+
```

12. Count the Records

```
>>> df.count()

4
```

Output

The RDDs and DataFrames were successfully created in Apache Spark. Transformations such as map(), filter(), and flatMap() were executed, and actions such as collect(), count(), and reduce() were successfully performed.

Result

Thus, Apache Spark was successfully used to create RDDs and DataFrames and perform various transformations and actions for processing large datasets.

Experiment 6: Data Preprocessing and Machine Learning using PySpark MLlib

Aim

Start Now

HADOOP:VERSION:1.0.1

To perform data preprocessing and implement classification or regression using Apache Spark MLlib pipelines.

Description

Apache Spark MLlib is a machine learning library used to build scalable machine learning applications.

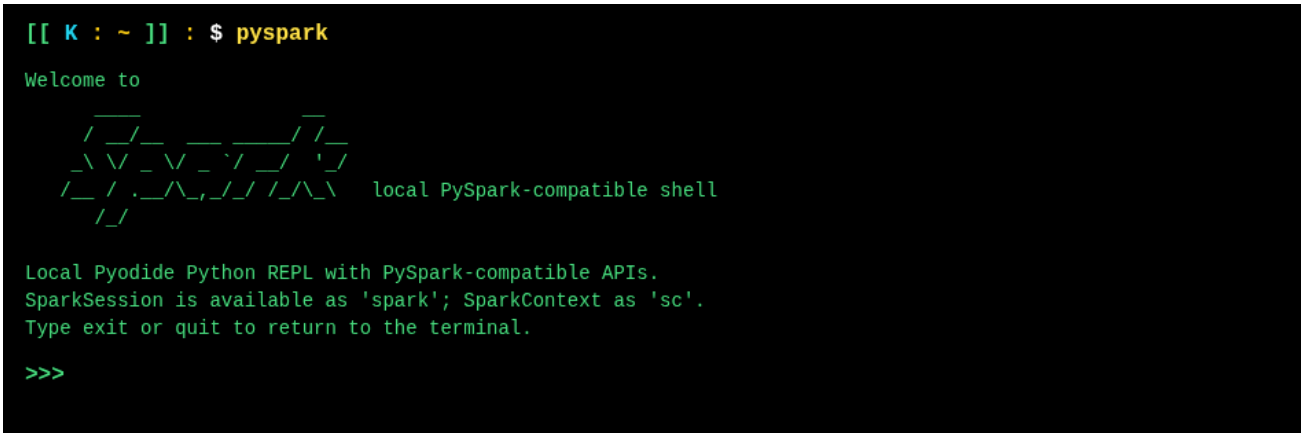
PySpark MLlib provides various tools for data preprocessing, feature extraction, classification, regression, and model evaluation.

In this experiment, PySpark MLlib is using for:

- Creating a Spark Session.
- Creating and loading datasets.
- Performing data preprocessing.
- Assembling features using VectorAssembler.
- Training a regression model.
- Making predictions .

Procedure

1. Start PySpark



2.Create DataSet

Start Now

```
>>> #creating data set
>>> data_set = [ (1,2) , (2,3) , (3,4) ]
```

3.Create SparkSession

```
>>> from pyspark.sql import SparkSession
>>> spark = SparkSession.builder.appName("ML_REGRESSION").master("local[*]").getOrCreate()
```

4.Create DataFrame

```
>>> df = spark.createDataFrame(data_set,['Input','Output'])
>>> df.show()
+-------+--------+
| Input | Output |
+-------+--------+
| 1     | 2      |
| 2     | 3      |
| 3     | 4      |
+-------+--------+
```

5.Create VectorAssembler

```
>>> from pyspark.ml.feature import VectorAssembler
>>> assembler = VectorAssembler(inputCols=["Input"],outputCol="vector")
>>> df = assembler.transform(df)
>>> df.show()
+-------+--------+--------+
| Input | Output | vector |
+-------+--------+--------+
| 1     | 2      | [1.0]  |
| 2     | 3      | [2.0]  |
| 3     | 4      | [3.0]  |
+-------+--------+--------+
```

6.Imort and create model [Regression]

```
>>> from pyspark.ml.regression import LinearRegression
>>> model = LinearRegression(featureCol="vector",labelCol="Output",maxIter=100)
```

7.Train & Display (m,c)

```
>>> model = model.fit(df)
>>> model.intercept
0.8599550994911589
>>> model.coefficients
[1.0616059920987337]
```

8.Create new Data Set

```
>>> new_data = [ (5,),(6,),(7,) ]

>>> new_data = spark.createDataFrame(new_data,["Input"])

>>> new_data.show()

+-------+
| Input |
+-------+
| 5     |
| 6     |
| 7     |
+-------+
```

9.Create new data vectors

```
>>> new_data = assembler.transform(new_data)

>>> new_data.show()

+-------+--------+
| Input | vector |
+-------+--------+
| 5     | [5.0]  |
| 6     | [6.0]  |
| 7     | [7.0]  |
+-------+--------+
```

10.Predict the values

```
>>> prediction = model.transform(new_data)

>>> prediction.show()

+-------+--------+------------------+
| Input | vector | prediction       |
+-------+--------+------------------+
| 5     | [5.0]  | 6.167985059984828 |
| 6     | [6.0]  | 7.229591052083561 |
| 7     | [7.0]  | 8.291197044182296 |
+-------+--------+------------------+
```

Output

The dataset was successfully created and preprocessed using PySpark MLlib. Features were assembled using VectorAssembler.

The Regression model was successfully trained using a machine learning . Predictions were generated and the model was evaluated .

Result

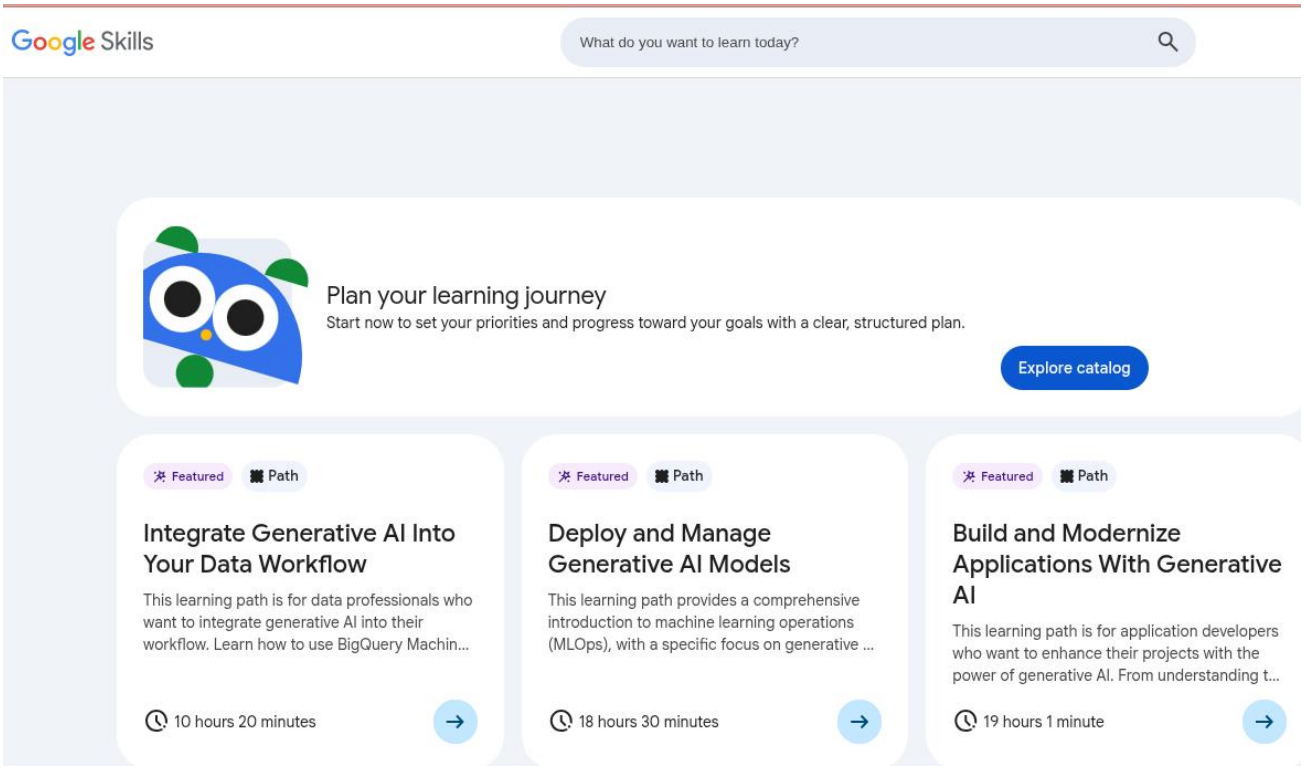
Thus, data preprocessing and machine learning were successfully implemented using PySpark MLlib. A regression model was trained using MLlib, predictions were generated successfully.

7-10 : cloud

Note : without proper credientials we can't use free resources . In this Course we go through the Google Cloud Services.

1.Create Account

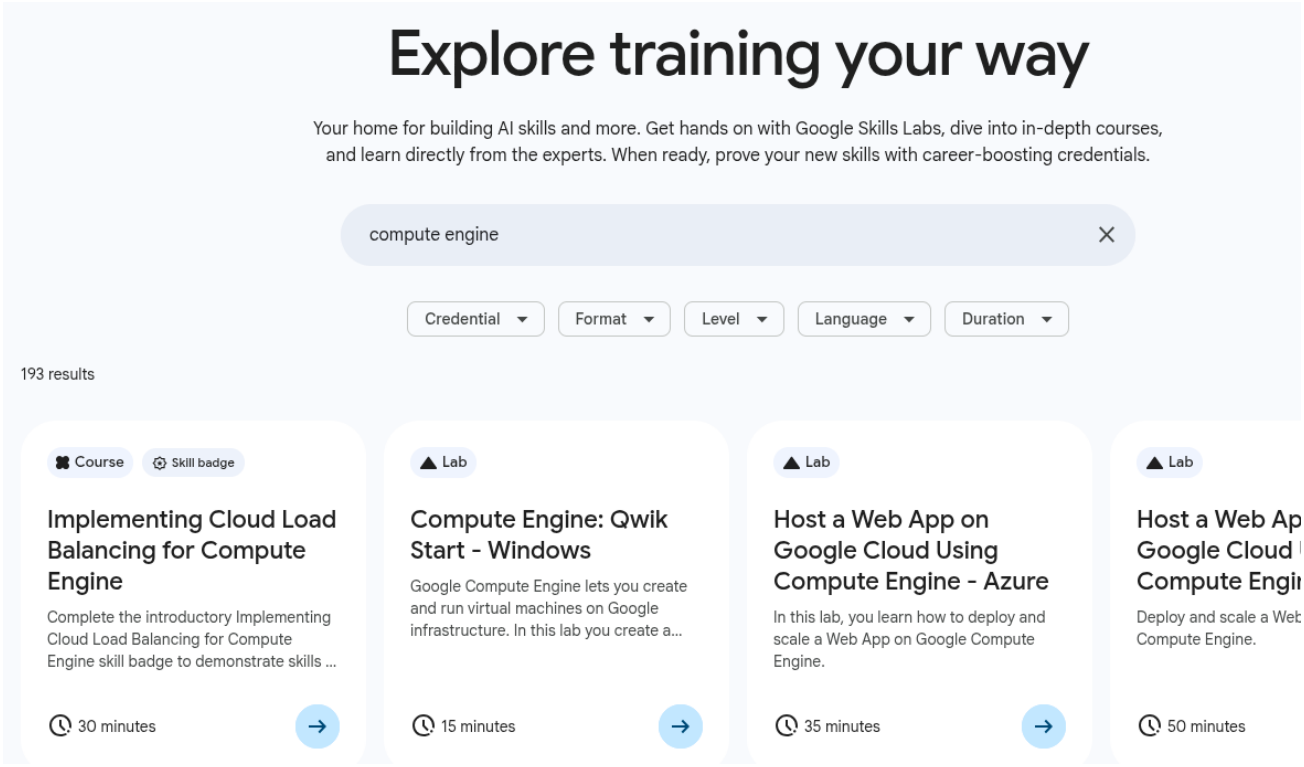
VISIT : https://www.skills.google/



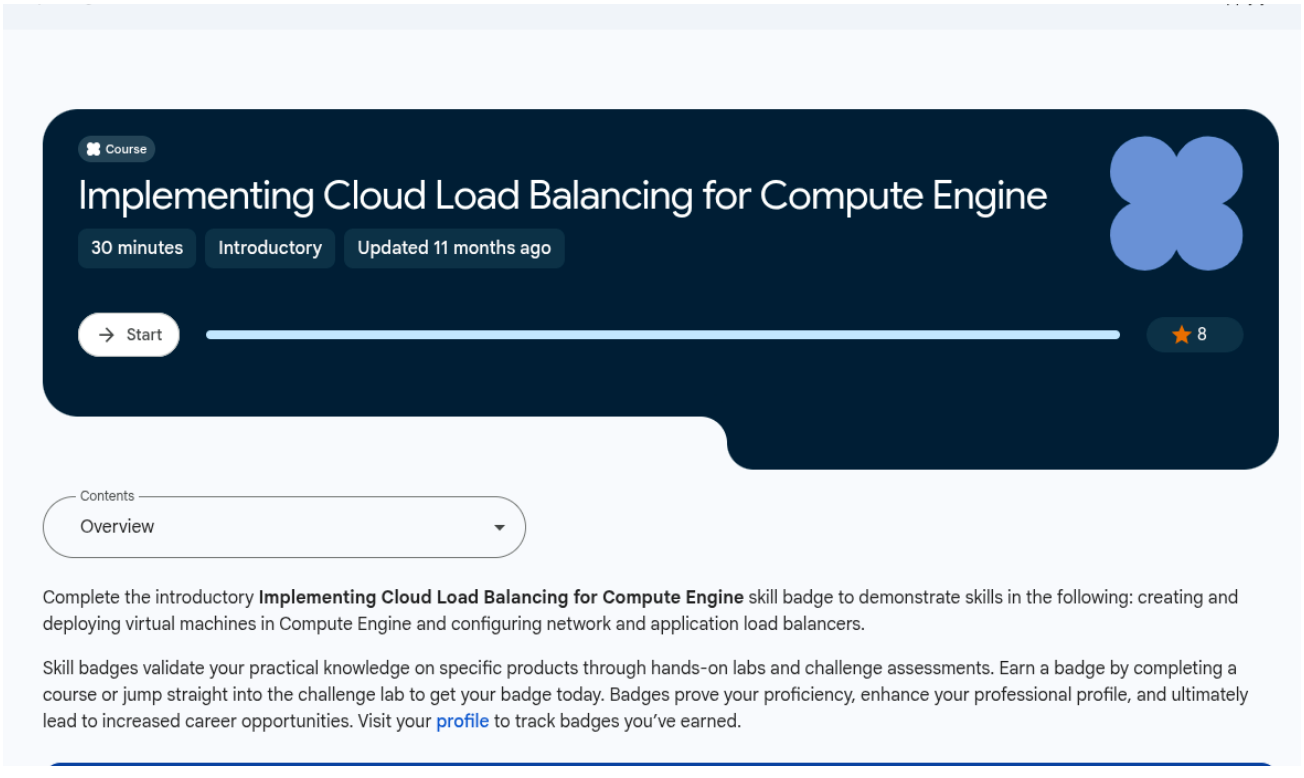
2.Search For Compute Engine

Start Now

HADOOP:VERSION:1.0.1



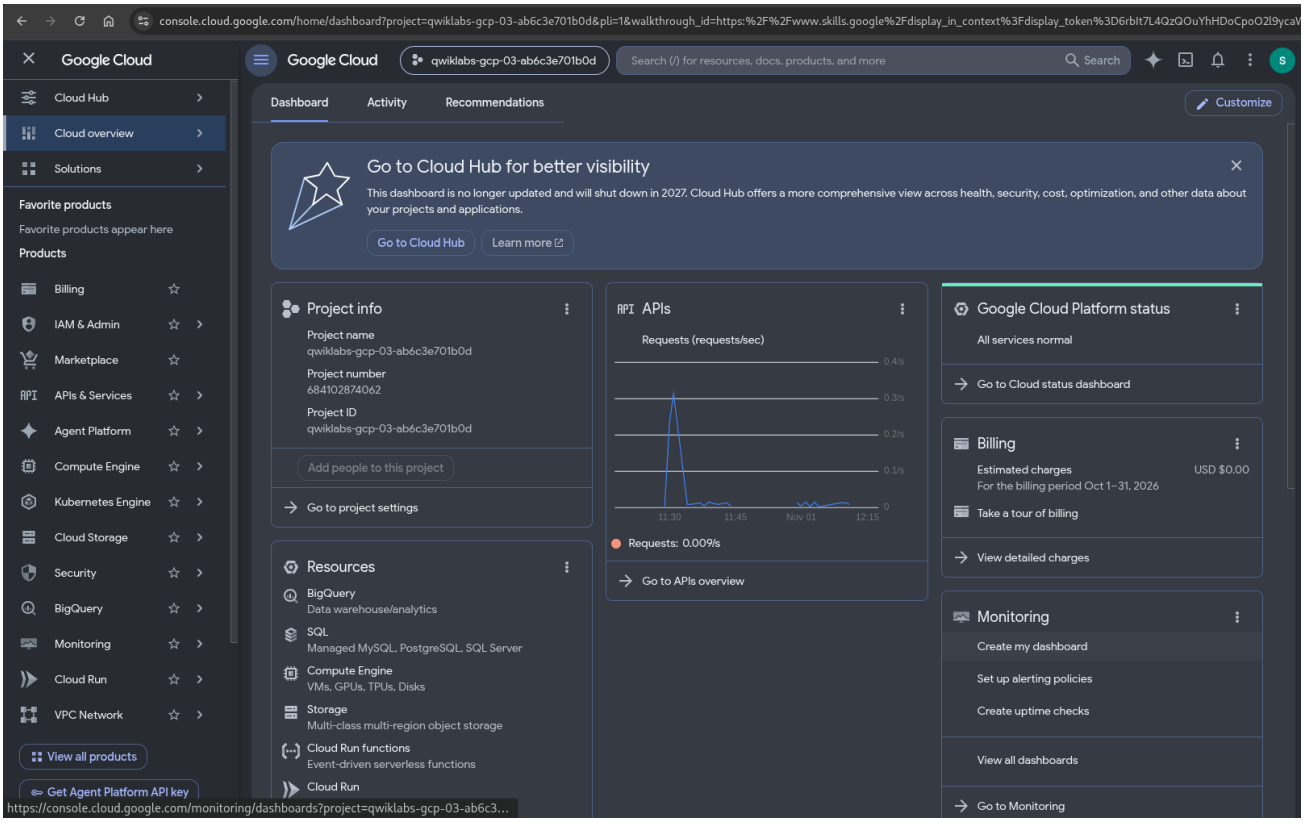
3.Go with Any One [no problem]



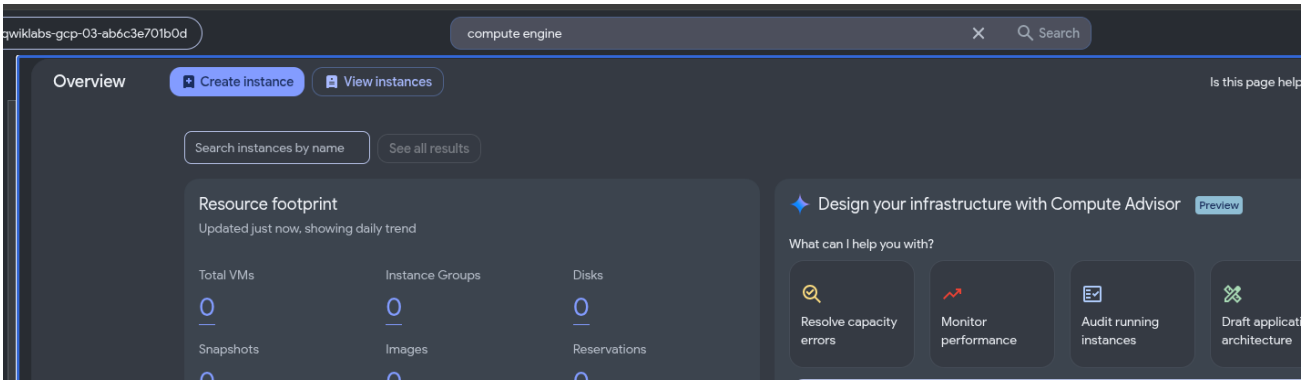
4.start the lab and ope the url in new cognitive window

Start Now

HADOOP:VERSION:1.0.1



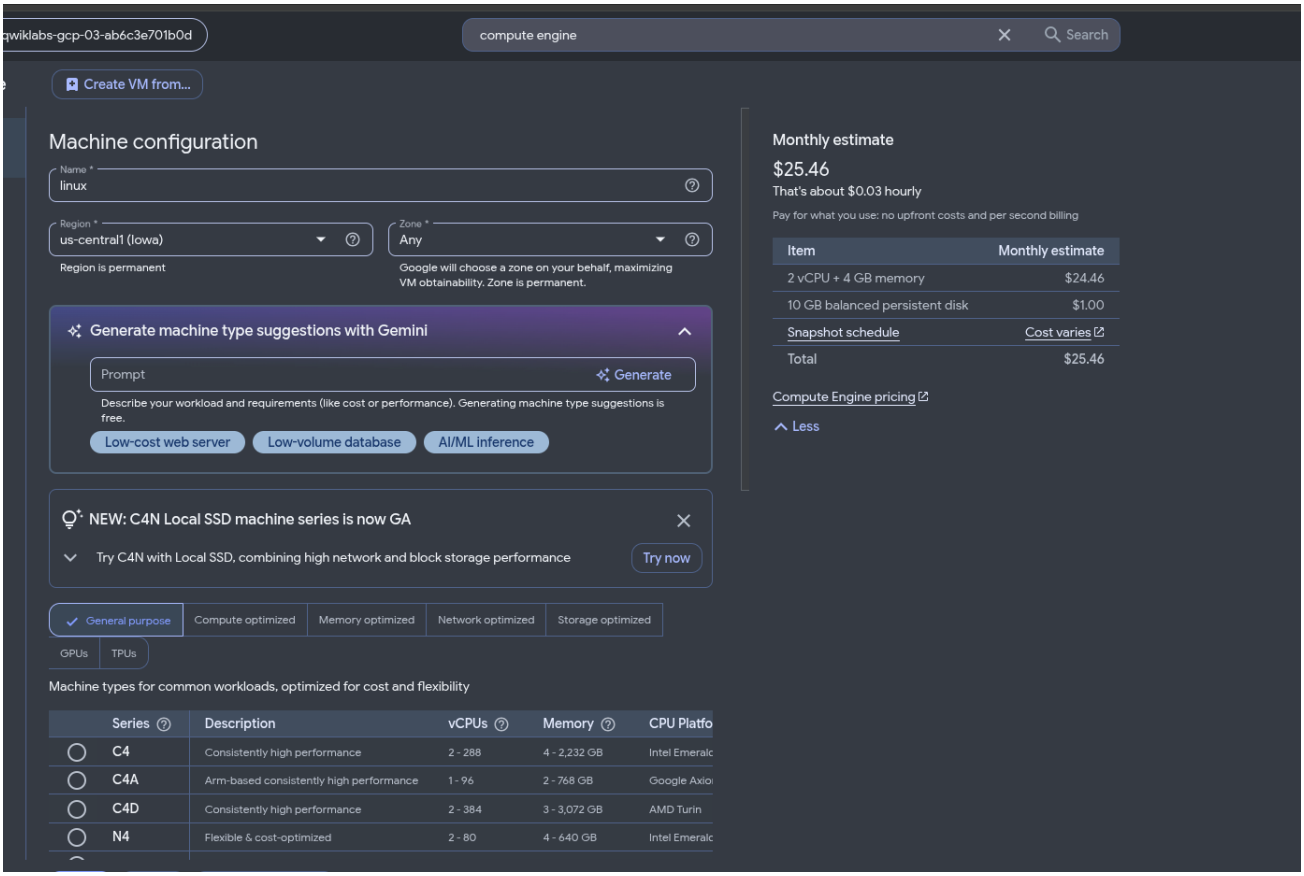
5.search for Compute Engine



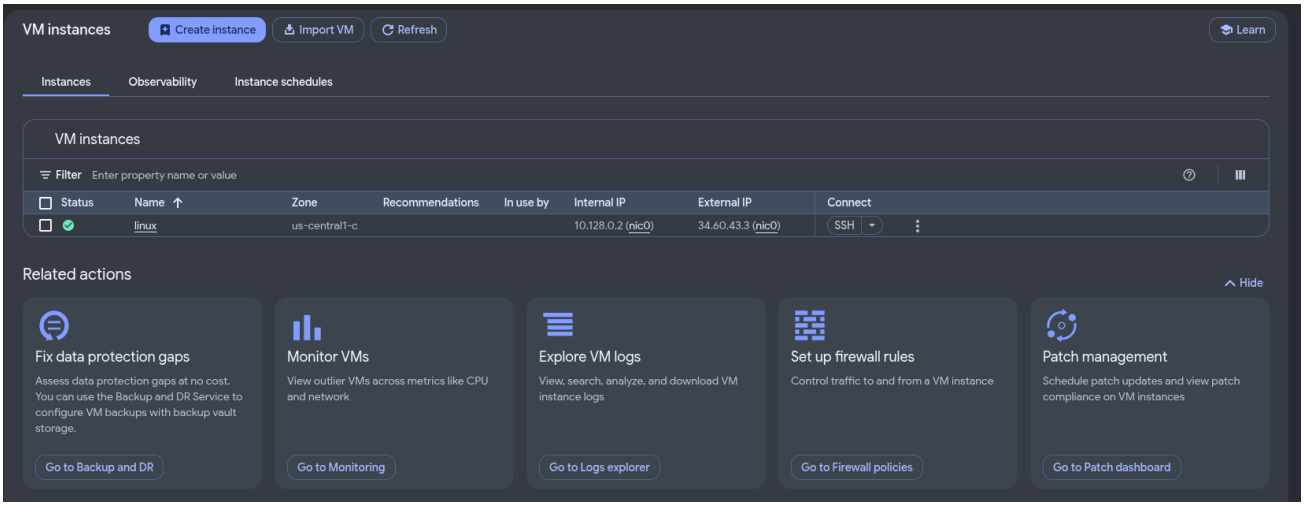
6.Click On Create Instance and provide some configurations

Start Now

HADOOP:VERSION:1.0.1



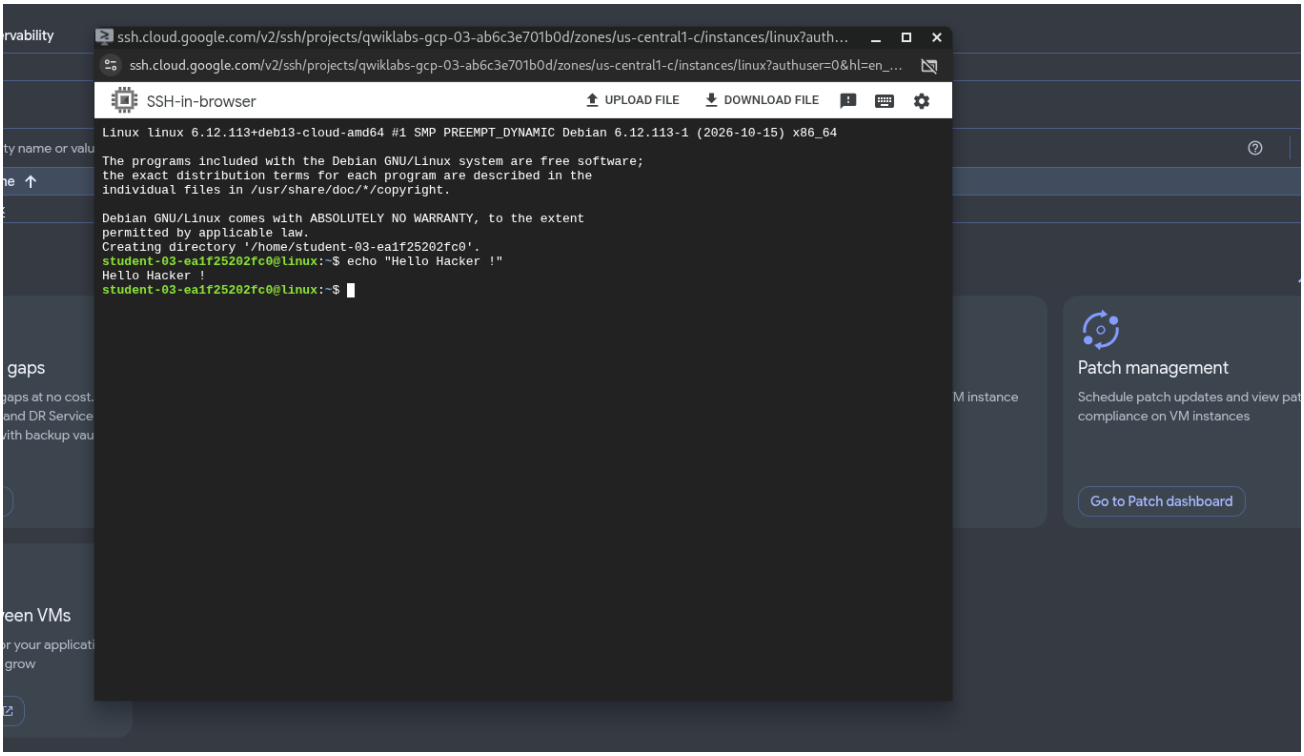
7.Tap on create button at bottom



8.Tap on SSH

Start Now

HADOOP:VERSION:1.0.1



Here we sucessfully created the Linux cloud VM by GCP, simillarly for all the cloud required examples / tasks follow the same process.

THANKING YOU !

Start Now